المذكرة الاحترافية

Software Engineering for Machine Learning Engineers

الفصل الثاني: SOLID Principles

خمس قواعد بتحدد إزاي تصمم كلاسات مرنة وقابلة للتوسع من غير ما تحتاج تعدل كود موجود

Single Responsibility • Open/Closed • Liskov Substitution • Interface Segregation • Dependency Inversion

جدول المحتويات

جدول المحتويات 2
مقدمة الفصل الثاني: من الكلاسات إلى مبادئ التصميم 5
٢،١ Single Responsibility Principle (SRP) 6
١. ليه مبدأ SRP مهم؟ 6
٢. المشكلة اللي بيحلها SRP 6
٣. إزاي كانت الشركات بتتعامل قبل تبني SRP بشكل واعي 6
٤. ليه الطريقة القديمة مش مناسبة مع النمو 6
٥. الفكرة الأساسية وراء SRP 6
٦. المفاهيم الأساسية 6
٧. المكونات: إزاي تحدد المسؤوليات 6
٨. إزاي المبدأ ده بيشتغل داخليًا 6
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 6
١٠. فين موقعه داخل مشروع ML حقيقي 7
١١. امتى تلتزم بـ SRP بصرامة 7
١٢. امتى ممكن تتساهل شوية 7
١٣. أفضل الممارسات 7
١٤. أشهر الأخطاء الشائعة 7
١٥. اعتبارات الأداء 7
١٦. الاعتبارات الأمنية 7
١٧. اعتبارات قابلية التوسع 7
١٨. مقارنة مع نهج بديل 8
١٩. مثال عملي صغير 8
٢٠. مثال من مشروع Production حقيقي 8
٢١. ملخص الموضوع 8
٢،٢ Open/Closed Principle (OCP) 9
١. ليه مبدأ OCP مهم؟ 9
٢. المشكلة اللي بيحلها OCP 9
٣. إزاي كانت الشركات بتتعامل قبل تبني OCP 9
٤. ليه الطريقة القديمة مش مناسبة 9
٥. الفكرة الأساسية وراء OCP 9
٦. المفاهيم الأساسية 9
٧. المكونات 9
٨. إزاي المبدأ ده بيشتغل داخليًا 9
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 10
١٠. فين موقعه داخل مشروع ML حقيقي 10
١١. امتى تطبق OCP 10
١٢. امتى OCP ممكن يكون مبالغ فيه 10
١٣. أفضل الممارسات 10
١٤. أشهر الأخطاء الشائعة 10
١٥. اعتبارات الأداء 10

١٦. الاعتبارات الأمنية 10
١٧. اعتبارات قابلية التوسع 10
١٨. مقارنة مع نهج بديل 11
١٩. مثال عملي صغير 11
٢٠. مثال من مشروع Production حقيقي 11
٢١. ملخص الموضوع 11
٢،٣ Liskov Substitution Principle (LSP) 12
١. ليه مبدأ LSP مهم؟ 12
٢. المشكلة اللي بيحلها LSP 12
٣. إزاي كانت الشركات بتتعامل قبل الانتباه لـ LSP 12
٤. ليه الطريقة القديمة مش مناسبة 12
٥. الفكرة الأساسية وراء LSP 12
٦. المفاهيم الأساسية 12
٧. المكونات 12
٨. إزاي المبدأ ده بيشتغل داخليًا 13
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 13
١٠. فين موقعه داخل مشروع ML حقيقي 13
١١. امتى تتنبه لـ LSP بشكل خاص 13
١٢. امتى ممكن تتجنب الوراثة تمامًا 13
١٣. أفضل الممارسات 13
١٤. أشهر الأخطاء الشائعة 13
١٥. اعتبارات الأداء 13
١٦. الاعتبارات الأمنية 13
١٧. اعتبارات قابلية التوسع 14
١٨. مقارنة مع نهج بديل 14
١٩. مثال عملي صغير 14
٢٠. مثال من مشروع Production حقيقي 14
٢١. ملخص الموضوع 14
٢،٤ Interface Segregation Principle (ISP) 15
١. ليه مبدأ ISP مهم؟ 15
٢. المشكلة اللي بيحلها ISP 15
٣. إزاي كانت الشركات بتتعامل قبل تبني ISP 15
٤. ليه الطريقة القديمة مش مناسبة 15
٥. الفكرة الأساسية وراء ISP 15
٦. المفاهيم الأساسية 15
٧. المكونات 15
٨. إزاي المبدأ ده بيشتغل داخليًا 15
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 16
١٠. فين موقعه داخل مشروع ML حقيقي 16
١١. امتى تطبق ISP 16
١٢. امتى ممكن تسيب واجهة واحدة موحدة 16

١٣. أفضل الممارسات 16
١٤. أشهر الأخطاء الشائعة 16
١٥. اعتبارات الأداء 16
١٦. الاعتبارات الأمنية 16
١٧. اعتبارات قابلية التوسع 17
١٨. مقارنة مع نهج بديل 17
١٩. مثال عملي صغير 17
٢٠. مثال من مشروع Production حقيقي 17
٢١. ملخص الموضوع 17
٢,٥ Dependency Inversion Principle (DIP) 18
١. ليه مبدأ DIP مهم؟ 18
٢. المشكلة اللي بيحلها DIP 18
٣. إزاي كانت الشركات بتتعامل قبل تبني DIP 18
٤. ليه الطريقة القديمة مش مناسبة 18
٥. الفكرة الأساسية وراء DIP 18
٦. المفاهيم الأساسية 18
٧. المكونات 18
٨. إزاي المبدأ ده بيشتغل داخليًا 19
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 19
١٠. فين موقعه داخل مشروع ML حقيقي 19
١١. امتى تطبق DIP 19
١٢. امتى DIP ممكن يكون زيادة عن اللزوم 19
١٣. أفضل الممارسات 19
١٤. أشهر الأخطاء الشائعة 19
١٥. اعتبارات الأداء 19
١٦. الاعتبارات الأمنية 19
١٧. اعتبارات قابلية التوسع 20
١٨. مقارنة مع نهج بديل 20
١٩. مثال عملي صغير 20
٢٠. مثال من مشروع Production حقيقي 20
٢١. ملخص الموضوع 20
مشروع عملي على الفصل: إعادة هيكلة نظام تدريب نماذج بمبادئ SOLID 22
خطوات التنفيذ 22
تحديات شائعة عند التطبيق 22
إزاي الفصل ده بيتستخدم داخل مشروع Machine Learning حقيقي 23
ملخص الفصل الثاني وربطه بالفصل القادم 23

مقدمة الفصل الثاني: من الكلاسات إلى مبادئ التصميم

في الفصل اللي فات اتكلمنا عن Classes كوحدة بتجمع الحالة (State) والسلوك (Behavior) في مكان واحد، وعرفنا مفاهيم زي Cohesion وCoupling وComposition. الفصل ده هيحول المفاهيم دي من مجرد إحساس عام بـ الكود الكويس لخمس قواعد دقيقة ومحددة اسمها SOLID، وهي اختصار لخمس مبادئ: Single Responsibility، Open/Closed، Liskov Substitution، Interface Segregation، وDependency Inversion.

SOLID مش نظرية أكاديمية بعيدة عن الواقع، دي القواعد اللي بتفسر ليه بعض الأنظمة سهل تضيفلها ميزة جديدة من غير ما تكسر حاجة، وليه أنظمة تانية أي تعديل بسيط فيها بيتسبب في كوارث غير متوقعة في أماكن تانية من الكود. الفرق بين النوعين دول غالبًا هو مدى الالتزام بمبادئ SOLID.

ليه بندرس الموضوع ده دلوقتي؟ لأن الفصل الجاي (Design Patterns) هو في الأساس مجموعة حلول جاهزة ومجربة لمشاكل تصميم متكررة، وكل نمط من الأنماط دي مبني فوق مبدأ أو أكتر من مبادئ SOLID. لو فهمت SOLID كويس، هتفهم Design Patterns كتطبيق طبيعي ليها، بدل ما تحفظها كقوالب جاهزة من غير فهم السبب وراها.

وهتحتاج SOLID كمان لما نوصل لـ Clean Architecture، لأن الطبقات (Layers) وقاعدة الاعتماد (Dependency Rule) اللي هنشرحها هناك هي في جوهرها تطبيق لمبدأ Dependency Inversion على مستوى النظام كله، مش بس على مستوى كلاس واحد.

٢,١ Single Responsibility Principle (SRP)

مبدأ المسؤولية الواحدة هو أول وأبسط مبادئ SOLID، وهو في الحقيقة صياغة رسمية لحاجة اتكلمنا عنها بشكل غير مباشر في فصل الـ Clean Code: كل كلاس لازم يكون له سبب واحد بس يخليه يتغير.

١. ليه مبدأ SRP مهم؟

لما كلاس واحد بيحمل أكتر من مسؤولية، أي تغيير في متطلبات إحدى المسؤوليات دي بيجبرك تعدل في الكلاس، وده بيزود احتمال إنك تكسر مسؤولية تانية من غير قصد. SRP بيقلل المخاطرة دي عن طريق فصل كل مسؤولية في كلاس مستقل.

في مشاريع ML، الالتزام بـ SRP بيسمح لفرق مختلفة (فريق البيانات، فريق النمذجة، فريق النشر) يشتغلوا على كلاسات منفصلة تمامًا من غير ما يتعارضوا مع بعض.

٢. المشكلة اللي بيحلها SRP

المشكلة هي الكلاسات اللي بتحمل مسؤوليات متعددة غير مترابطة، زي كلاس ModelManager بيدرب النموذج، ويحفظه في قاعدة بيانات، ويرسل تقرير بالإيميل. أي تغيير في طريقة الإرسال بالإيميل بيجبرك تفتح وتعدل نفس الكلاس المسؤول عن التدريب، وده بيزود فرصة كسر منطق التدريب بالخطأ.

٣. إزاي كانت الشركات بتتعامل قبل تبني SRP بشكل واعي

في المشاريع المبكرة والنماذج الأولية (Prototypes)، كان شائع تجميع كل المنطق المرتبط بمهمة معينة في كلاس واحد كبير عشان السرعة، من غير التفكير في فصل المسؤوليات، خصوصًا لما الفريق صغير ومفيش احتكاك حقيقي بين المسؤوليات المختلفة.

٤. ليه الطريقة القديمة مش مناسبة مع النمو

مع نمو المشروع وزيادة عدد المهندسين، الكلاس متعدد المسؤوليات بيتحول لنقطة اختناق (Bottleneck): أكتر من مهندس بيحتاجوا يعدلوا في نفس الملف لأسباب مختلفة تمامًا، وده بيزود تعارضات الدمج (Merge Conflicts) واحتمالية كسر الكود.

٥. الفكرة الأساسية وراء SRP

الفكرة إن كل كلاس لازم يكون مسؤول عن جزء واحد من وظيفة النظام، ولو حد سألك ده الكلاس بيعمل إيه واحتجت تستخدم كلمة و في إجابتك، ده علامة إنه محتاج ينقسم. السبب في التغيير (Reason to Change) لازم يكون واحد بس لكل كلاس.

٦. المفاهيم الأساسية

- Reason to Change: كل كلاس لازم يكون له سبب واحد بس يخليه يتعدل.
- Separation of Concerns: فصل الاهتمامات المختلفة (بيانات، منطق أعمال، عرض) في وحدات منفصلة.
- Cohesion: العناصر جوه الكلاس الواحد لازم تكون مرتبطة منطقيًا ببعض، وهو مقياس مباشر لمدى التزام الكلاس بـ SRP.

٧. المكونات: إزاي تحدد المسؤوليات

- حدد الأسباب المحتملة للتغيير في الكلاس (تغيير في منطق العمل، تغيير في طريقة التخزين، تغيير في طريقة العرض).
- لو أكتر من سبب موجود، افصل كل سبب في كلاس مستقل.
- استخدم التركيب (Composition) لربط الكلاسات المنفصلة دي مع بعض عند الحاجة.

٨. إزاي المبدأ ده بيشتغل داخليًا

لما تفصل المسؤوليات، بتقلل من عدد الأسباب اللي ممكن تخلي الكود يتغير، وده بيقلل بشكل مباشر تكرار فتح وتعديل نفس الملف. من الناحية العملية، ده بيترجم لتقليل في عدد الـ Merge Conflicts وتقليل في نطاق تأثير أي Bug جديد (Blast Radius).

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في فرق الـ ML Engineering، بتلاقي فصل واضح بين كلاس مسؤول عن الوصول للبيانات (Data Access Layer)، وكلاس مسؤول عن منطق النموذج (Model Logic)، وكلاس مسؤول عن الـ Serving (تقديم النموذج للمستخدمين). كل فريق بيمتلك جزء محدد، وده بيقلل الاحتكاك بين الفرق المختلفة.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في الفصل بين كلاس DataLoader وكلاس FeatureEngineer وكلاس ModelTrainer وكلاس ModelEvaluator، كل واحد فيهم مستقل تمامًا وله مسؤولية واحدة واضحة.

١١. امتى تلتزم بـ SRP بصرامة

- لما الكلاس هيتطور ويتعدل باستمرار مع الوقت.
- لما فرق مختلفة هتحتاج تتعامل مع أجزاء مختلفة من نفس النظام.

١٢. امتى ممكن تتساهل شوية

- في سكربت صغير جدًا ومؤقت هيتشغل مرة واحدة وينتهي دوره.
- لما الفصل المفرط للمسؤوليات هيزود تعقيد غير ضروري لمشروع بسيط جدًا.

١٣. أفضل الممارسات

Best Practices

- اسأل نفسك دايمًا ده الكلاس بيعمل إيه، ولو الإجابة فيها كلمة و، فكر في التقسيم.
- افصل منطق الوصول للبيانات عن منطق الأعمال عن منطق العرض.
- استخدم التركيب (Composition) للربط بين الكلاسات المنفصلة بدل دمجهم في كلاس واحد.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- بناء كلاس واحد بيدير كل شيء (God Class) بحجة السرعة في التطوير.
- الخلط بين منطق الأعمال ومنطق التخزين أو العرض في نفس الكلاس.
- المبالغة في تطبيق SRP لدرجة تقسيم كل شيء لكلاسات صغيرة جدًا من غير داعي حقيقي (Over-engineering).

١٥. اعتبارات الأداء

فصل المسؤوليات لكلاسات متعددة بيضيف تكلفة بسيطة جدًا في استدعاء الدوال بين الكلاسات، لكنها غالبًا مهملة مقارنة بفايدة سهولة الصيانة والاختبار.

١٦. الاعتبارات الأمنية

فصل المسؤوليات بيسمح بتطبيق قواعد أمان مختلفة على كل جزء حسب حساسيته، زي تطبيق تحقق صارم من الصلاحيات على كلاس الوصول للبيانات الحساسة، بدون ما يتأثر منطق العرض أو الحساب.

١٧. اعتبارات قابلية التوسع

الكلاسات ذات المسؤولية الواحدة أسهل بكتير في التوسع الأفقي (Horizontal Scaling)، لأنك تقدر تستبدل أو تحسن جزء واحد (زي طريقة تخزين البيانات) من غير التأثير على باقي النظام.

١٨. مقارنة مع نهج بديل

البديل هو الكلاس الشامل (All-in-One Class) اللي بيوفر وقت في البداية لأنك مش محتاج تنظم العلاقات بين كذا كلاس. لكن الأسلوب ده بيفشل بسرعة مع نمو المشروع، والشركات الكبرى بتفضل دايمًا الفصل الواضح رغم التكلفة الإضافية البسيطة في التنظيم الأولي.

١٩. مثال عملي صغير

```
# Before - one class doing training, storage, and notification
class ModelManager:
    def train(self, data):
        pass
    def save_to_database(self, model):
        pass
    def send_email_report(self, metrics):
        pass

# After - each responsibility in its own class
class ModelTrainer:
    def train(self, data):
        pass

class ModelRepository:
    def save(self, model):
        pass

class TrainingReporter:
    def send_email_report(self, metrics):
        pass
```

٢٠. مثال من مشروع Production حقيقي

في نظام حقيقي لتوصية المحتوى، فريق الـ Feature Store مسؤول بس عن كلاس FeatureRetriever، وفريق النمذجة مسؤول عن كلاس RankingModel، وفريق الـ Infrastructure مسؤول عن كلاس ModelServer. الفصل ده بيسمح لكل فريق يشتغل ويطور بسرعته الخاصة من غير انتظار الفرق التانية.

٢١. ملخص الموضوع

خلاصة SRP

- كل كلاس لازم يكون له سبب واحد بس يخليه يتغير.
- افصل منطق الأعمال عن التخزين عن العرض في كلاسات مستقلة.
- الفصل الواضح بيقلل تعارضات الدمج ونطاق تأثير الأخطاء.
- احذر من المبالغة في التقسيم لدرجة تعقيد المشروع من غير داعي.

٢,٢ Open/Closed Principle (OCP)

المبدأ ده بيقول إن الكيانات البرمجية (كلاسات، دوال، وحدات) لازم تكون مفتوحة للتوسع (Open for Extension) لكن مغلقة للتعديل (Closed for Modification). يعني تقدر تضيف سلوك جديد من غير ما تلمس الكود الموجود بالفعل.

١. ليه مبدأ OCP مهم؟

كل مرة تعدل في كود شغال بالفعل، بتخاطر بكسر سلوك موجود ومُختبر. OCP بيقلل المخاطرة دي عن طريق تصميم الكود بحيث الإضافات الجديدة بتحصل عن طريق كود جديد، مش عن طريق تعديل كود قديم.

في مشاريع ML، ده مهم جدًا لما تحتاج تضيف خوارزمية جديدة أو مصدر بيانات جديد من غير ما تخاطر بكسر الخوارزميات أو المصادر الموجودة بالفعل واللي شغالة في Production.

٢. المشكلة اللي بيحلها OCP

المشكلة هي الكود اللي بيعتمد على سلسلة شروط if/elif طويلة عشان يقرر أي منطق ينفذ حسب النوع، وكل ما تضيف نوع جديد، لازم تفتح نفس الدالة وتضيف شرط جديد. ده بيزود احتمال كسر الأنواع الموجودة بالفعل، وبيخلي الدالة تكبر بلا حدود مع الوقت.

٣. إزاي كانت الشركات بتتعامل قبل تبني OCP

كان شائع استخدام شروط if/else أو switch statements كبيرة للتعامل مع الأنواع المختلفة من البيانات أو النماذج، وكل ما ظهر نوع جديد، المهندس كان بيرجع لنفس الدالة القديمة ويضيف حالة جديدة ليها.

٤. ليه الطريقة القديمة مش مناسبة

كل تعديل في دالة الشروط الطويلة دي بيحمل خطر كسر الحالات الموجودة، وبمرور الوقت الدالة بتتحول لكتلة ضخمة من الشروط المتشابكة صعبة الفهم والاختبار. كمان أي فريق تاني يحتاج يضيف نوع جديد لازم يعدل في نفس الملف المركزي، وده بيزود التعارضات.

٥. الفكرة الأساسية وراء OCP

بدل الشروط المتعددة، بتعرف واجهة (Interface) أو كلاس أساسي مجرد (Abstract Base Class)، وكل نوع جديد بيتنفذ كتطبيق منفصل ليها. الكود اللي بيستخدم الواجهة دي مش بيحتاج يعرف تفاصيل كل نوع، وبالتالي إضافة نوع جديد مش محتاجة تعديل في الكود الموجود، بس إضافة كلاس جديد.

٦. المفاهيم الأساسية

- Abstraction: تعريف واجهة عامة تصف السلوك المطلوب من غير تحديد التفاصيل.
- Polymorphism: القدرة على استدعاء نفس الواجهة على أنواع مختلفة والحصول على سلوك مختلف حسب النوع.
- Plugin Architecture: تصميم النظام بحيث الإضافات الجديدة (Plugins) بتتضاف من غير تعديل الجوهر الأساسي للنظام.

٧. المكونات

- واجهة أو كلاس مجرد يحدد العقد (Contract) المطلوب من كل تطبيق.
- تطبيقات متعددة (Concrete Implementations) لكل حالة أو نوع.
- كود مستخدم (Client Code) بيتعامل مع الواجهة بس، من غير معرفة التفاصيل الداخلية لكل تطبيق.

٨. إزاي المبدأ ده بيشتغل داخليًا

لما الكود المستخدم بيتعامل مع واجهة مجردة بدل كلاسات محددة، فهو بيعتمد على Polymorphism عشان يقرر السلوك الفعلي وقت التشغيل (Runtime) حسب النوع الممرر له. إضافة نوع جديد بيبقى بمثابة إضافة تطبيق جديد للواجهة، من غير أي حاجة تتغير في الكود المستخدم نفسه.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في مكتبات ML الكبرى زي scikit-learn، كل خوارزمية (RandomForest، LinearRegression، وغيرها) بتلتزم بنفس الواجهة الأساسية (fit وpredict)، وده بيسمح بإضافة خوارزميات جديدة للمكتبة من غير التأثير على الكود اللي بيستخدم الخوارزميات الموجودة.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في تصميم أنظمة تدعم أكتر من نوع نموذج (Model Type) أو أكتر من مصدر بيانات (Data Source)، حيث كل نوع بيتنفذ كتطبيق منفصل لواجهة موحدة، والـ Pipeline الرئيسي بيتعامل مع الواجهة دي بس.

١١. امتى تطبق OCP

- لما تتوقع إضافة أنواع جديدة (خوارزميات، مصادر بيانات، تنسيقات) بشكل متكرر مستقبلًا.
- لما عندك أكتر من فريق بيحتاج يضيف تطبيقات جديدة من غير التنسيق المستمر مع فريق مركزي.

١٢. امتى OCP ممكن يكون مبالغ فيه

- لما عدد الأنواع محدود وثابت ومش متوقع يتغير، فبناء طبقة تجريد كاملة بيزود التعقيد من غير فايدة حقيقية.
- في كود تجريبي سريع مش هيتطور لاحقًا.

١٣. أفضل الممارسات

Best Practices

- استخدم Abstract Base Classes أو Protocols في Python لتعريف الواجهة المشتركة.
- خلي كود المستخدم (Client Code) يتعامل مع الواجهة المجردة بس، مش الأنواع المحددة.
- لا تطبق OCP إلا لو فيه توقع حقيقي إن أنواع جديدة هتضاف مستقبلًا.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- الاستمرار في إضافة شروط if/elif لكل نوع جديد بدل تعريف واجهة مشتركة.
- بناء طبقة تجريد معقدة جدًا لحالات بسيطة ونادرة التغيير (Over-engineering).
- كسر مبدأ OCP عن طريق فحص نوع الكائن (Type Checking) داخل الكود المستخدم بدل الاعتماد على Polymorphism.

١٥. اعتبارات الأداء

استخدام الـ Polymorphism بدل الشروط المباشرة عادة بيكون له تأثير مهمل على الأداء في لغات زي Python، لأن استدعاء Method عن طريق واجهة مش أبطأ بشكل ملحوظ عن استدعاء مباشر.

١٦. الاعتبارات الأمنية

تصميم قائم على واجهات واضحة بيسهل فرض قواعد تحقق موحدة على كل التطبيقات الجديدة، لأن أي تطبيق جديد لازم يلتزم بنفس العقد (Contract) بما فيه أي متطلبات أمان محددة في الواجهة.

١٧. اعتبارات قابلية التوسع

OCP هو أساس قابلية التوسع الحقيقية: النظام المصمم صح بيسمح بإضافة قدرات جديدة (نماذج، مصادر بيانات، قنوات إشعارات) من غير الحاجة لتعديل الكود الأساسي المختبر بالفعل، وده بيقلل المخاطرة مع كل توسعة جديدة.

١٨. مقارنة مع نهج بديل

البديل هو التعديل المباشر في الكود الموجود لإضافة كل حالة جديدة (شروط if/elif متتالية). الأسلوب ده أسرع لحالة واحدة أو اتنين، لكنه بيتحول لعبء كبير مع تزايد عدد الحالات، والشركات الكبرى بتفضل الاستثمار في طبقة تجريد واضحة من البداية لو التوسع المستقبلي متوقع.

١٩. مثال عملي صغير

```
# Before - adding a new model type requires editing this function
def train_model(model_type, data):
    if model_type == 'linear':
        return LinearRegression().fit(data['X'], data['y'])
    elif model_type == 'tree':
        return DecisionTreeRegressor().fit(data['X'], data['y'])
    # adding a new type means modifying this function again

# After - open for extension, closed for modification
from abc import ABC, abstractmethod

class ModelStrategy(ABC):
    @abstractmethod
    def train(self, data):
        ...

class LinearModelStrategy(ModelStrategy):
    def train(self, data):
        return LinearRegression().fit(data['X'], data['y'])

class TreeModelStrategy(ModelStrategy):
    def train(self, data):
        return DecisionTreeRegressor().fit(data['X'], data['y'])

# Adding a new model type only means adding a new class,
# with zero changes to existing code.
```

٢٠. مثال من مشروع Production حقيقي

في نظام Notification حقيقي مرتبط بنتائج نموذج ML (زي تنبيه عند اكتشاف احتيال)، النظام بيدعم قنوات إشعار متعددة (Email، Slack، SMS) عن طريق واجهة موحدة NotificationChannel. إضافة قناة جديدة (زي WhatsApp) بتتم بإضافة كلاس جديد بس، من غير أي تعديل في منطق اكتشاف الاحتيال نفسه.

٢١. ملخص الموضوع

خلاصة OCP

- الكود لازم يكون مفتوح للتوسع لكن مغلق للتعديل.
- استخدم واجهات مجردة وPolymorphism بدل سلاسل الشروط الطويلة.
- طبق المبدأ ده بس لما التوسع المستقبلي متوقع فعلًا، تجنبًا للتعقيد الزائد.

٢,٣ Liskov Substitution Principle (LSP)

المبدأ ده بيقول إن أي كلاس فرعي (Subclass) لازم يقدر يحل محل الكلاس الأساسي (Base Class) بتاعه من غير ما يكسر صحة البرنامج. يعني لو عندك دالة بتستقبل الكلاس الأساسي، أي كلاس فرعي منه لازم يشتغل صح لو استخدمته بدل الأساسي.

يعني كل التصرفات بتاعت الكلاس الأب لازم كلاس الأبن يعملها

١. ليه مبدأ LSP مهم؟

الوراثة (Inheritance) في البرمجة الكائنية بتوحي بعلاقة is-a، يعني الكلاس الفرعي هو نوع من الكلاس الأساسي. لو الكلاس الفرعي بيخالف توقعات الكلاس الأساسي (زي إنه يرمي خطأ في حالة الأساسي كان بيرجع قيمة عادية)، فالعلاقة is-a دي بتبقى وهمية، وأي كود بيعتمد على الكلاس الأساسي ممكن ينكسر لما يستخدم الكلاس الفرعي من غير سابق إنذار.

في مشاريع ML، ده بيظهر لما يكون عندك كلاس أساسي BaseModel وكلاسات فرعية بتمثل نماذج مختلفة؛ لازم كل نموذج فرعي يلتزم بنفس السلوك المتوقع من الواجهة الأساسية.

٢. المشكلة اللي بيحلها LSP

المشكلة هي الوراثة السطحية اللي بتعتمد بس على تشابه الاسم أو جزء من السلوك، من غير ضمان إن الكلاس الفرعي بيحترم كل توقعات الكلاس الأساسي. مثال كلاسيكي: كلاس Square (مربع) بيرث من كلاس Rectangle (مستطيل)، لكن تغيير عرض المربع لازم يغير طوله كمان، وده بيخالف توقع إن تغيير العرض في المستطيل مايأثرش على الطول.

٣. إزاي كانت الشركات بتتعامل قبل الانتباه لـ LSP

كان شائع استخدام الوراثة بشكل حرفي بناءً على التشابه الشكلي بين الكيانات (زي المربع والمستطيل هندسيًا)، من غير التحقق من إن السلوك الفعلي متوافق مع العقد المتوقع من الكلاس الأساسي.

٤. ليه الطريقة القديمة مش مناسبة

لما كلاس فرعي بيخالف توقعات الكلاس الأساسي، أي كود بيتعامل مع الكلاس الأساسي بشكل عام (Polymorphic Code) ممكن يفشل بشكل غير متوقع لما يستقبل الكلاس الفرعي ده، وده بيخلق أخطاء صعبة التتبع لأنها بتظهر بس في حالات استخدام معينة.

٥. الفكرة الأساسية وراء LSP

الفكرة إن الوراثة لازم تعتمد على السلوك المتوقع مش بس الشكل الظاهري. قبل ما تخلي كلاس يرث من كلاس تاني، اسأل نفسك: هل الكلاس الفرعي ده هيحترم كل الوعود (Contracts) اللي الكلاس الأساسي بيقطعها؟ لو الإجابة لأ، فالتركيب (Composition) غالبًا خيار أفضل من الوراثة هنا.

٦. المفاهيم الأساسية

- Behavioral Subtyping: الكلاس الفرعي لازم يحترم نفس السلوك المتوقع من الكلاس الأساسي، مش بس نفس الشكل.
- Preconditions and Postconditions: الشروط المطلوبة قبل وبعد تنفيذ Method؛ الكلاس الفرعي ميقدرش يشدد الشروط المطلوبة قبل التنفيذ (Preconditions) ولا يخفف الضمانات بعد التنفيذ (Postconditions).
- Design by Contract: فكرة إن كل Method بيمثل عقد واضح، والكلاسات الفرعية لازم تحترم العقد ده بالكامل.

٧. المكونات

- الكلاس الأساسي (Base Class) بيحدد العقد المتوقع (المدخلات، المخرجات، الحالات الاستثنائية).
- الكلاس الفرعي (Subclass) لازم يلتزم بنفس العقد أو يوسعه من غير ما يخالفه.
- اختبارات مشتركة (Shared Test Suite) ممكن تتطبق على كل الكلاسات الفرعية للتأكد من التزامها بالعقد.

٨. إزاي المبدأ ده بيشتغل داخليًا

الكود اللي بيستخدم الكلاس الأساسي بيتوقع سلوك معين، وأي انحراف عن التوقع ده في الكلاس الفرعي بيظهر بس وقت التشغيل الفعلي (Runtime)، مش وقت التصريف، وده اللي بيخلي مشاكل LSP صعبة الاكتشاف المبكر ولازم تتفحص بعناية في مرحلة تصميم الكلاسات الفرعية.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في أنظمة ML الكبيرة، أي كلاس نموذج جديد بيرث من BaseModel لازم يمر باختبارات موحدة (Contract Tests) تتأكد إنه بيلتزم بنفس الواجهة والسلوك المتوقع (زي إرجاع نفس شكل المخرجات) قبل ما يتقبل في الـ Codebase الرئيسي.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في أي هيكل وراثة (Class Hierarchy) للنماذج أو معالجات البيانات، حيث كل نوع فرعي لازم يشتغل بشكل متبادل مع باقي الأنواع من غير ما يكسر الكود اللي بيستخدمهم بشكل عام.

١١. امتى تنتبه لـ LSP بشكل خاص

- لما تبني هيكل وراثة (Inheritance Hierarchy) لأنواع متعددة من نفس المفهوم.
- لما كود مشترك بيتعامل مع الكلاسات دي بشكل عام (Polymorphic) من غير معرفة النوع الدقيق.

١٢. امتى ممكن تتجنب الوراثة تمامًا

- لما العلاقة بين الكلاسات مش علاقة is-a حقيقية، حتى لو فيه تشابه سطحي.
- في الحالات دي، التركيب (Composition) أو الواجهات (Interfaces/Protocols) بديل أأمن من الوراثة.

١٣. أفضل الممارسات

Best Practices

- قبل استخدام الوراثة، تأكد إن العلاقة is-a حقيقية على مستوى السلوك مش بس الشكل.
- اكتب اختبارات مشتركة (Contract Tests) تتطبق على كل الكلاسات الفرعية للتأكد من التزامها بالعقد.
- لو محتاج تخالف سلوك الكلاس الأساسي، فكر في التركيب بدل الوراثة.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- استخدام الوراثة بناءً على تشابه شكلي بين كيانين من غير التحقق من توافق السلوك.
- كلاس فرعي بيرمي استثناء (Exception) في حالة الكلاس الأساسي كان بيرجع قيمة عادية.
- كلاس فرعي بيتجاهل جزء من وظيفة الكلاس الأساسي أو بيغير معنى المدخلات المتوقعة.

١٥. اعتبارات الأداء

الالتزام بـ LSP مالوش تأثير مباشر على الأداء، لكن الأخطاء الناتجة عن مخالفته (سلوك غير متوقع وقت التشغيل) ممكن تسبب معالجة استثناءات إضافية أو إعادة محاولات (Retries) بتؤثر على الأداء بشكل غير مباشر.

١٦. الاعتبارات الأمنية

كلاس فرعي بيخالف توقعات الأمان المفروضة في الكلاس الأساسي (زي التحقق من الصلاحيات) ممكن يفتح ثغرة أمنية خطيرة، لأن الكود اللي بيستخدم الكلاس الأساسي بيفترض إن كل الكلاسات الفرعية بتلتزم بنفس قواعد الأمان.

١٧. اعتبارات قابلية التوسع

هيكل وراثة ملتزم بـ LSP بيسمح بإضافة أنواع جديدة بثقة، لأن أي كود موجود بيتعامل مع الكلاس الأساسي هيشتغل صح تلقائيًا مع أي نوع جديد ملتزم بالعقد، من غير الحاجة لاختبار كل حالة استخدام يدويًا.

١٨. مقارنة مع نهج بديل

البديل هو فحص نوع الكائن (Type Checking) يدويًا قبل التعامل معاه، للتحايل على عدم التوافق السلوكي. الأسلوب ده بيخالف فكرة الـ Polymorphism من أساسها وبيرجع الكود لسلاسل شروط طويلة تشبه المشكلة اللي OCP بيحلها. الحل الأفضل دايمًا هو ضمان التوافق السلوكي من الأساس بدل التحايل عليه لاحقًا.

١٩. مثال عملي صغير

```
# Before - subclass breaks the base class contract
class Bird:
    def fly(self):
        return 'Flying'

class Penguin(Bird):
    def fly(self):
        raise NotImplementedError('Penguins cannot fly')

# After - restructure the hierarchy to match real behavior
class Bird:
    def move(self):
        ...

class FlyingBird(Bird):
    def move(self):
        return 'Flying'

class Penguin(Bird):
    def move(self):
        return 'Swimming'
```

٢٠. مثال من مشروع Production حقيقي

في نظام يدعم أنواع نماذج متعددة عن طريق كلاس أساسي BaseModel بيوفر method اسمه predict() بيرجع دايمًا مصفوفة من الاحتمالات (Probabilities). أي نموذج فرعي جديد لازم يلتزم بنفس الشكل ده بالظبط، حتى لو الخوارزمية الداخلية مختلفة تمامًا، عشان كود الـ Serving اللي بيستخدم predict() يشتغل صح مع أي نموذج من غير تعديل.

٢١. ملخص الموضوع

خلاصة LSP

- الكلاس الفرعي لازم يقدر يحل محل الكلاس الأساسي من غير ما يكسر صحة البرنامج.
- الوراثة لازم تعتمد على توافق السلوك، مش بس التشابه الشكلي.
- لو السلوك مش متوافق، استخدم التركيب بدل الوراثة.

٢,٤ Interface Segregation Principle (ISP)

المبدأ ده بيقول إنه من الأفضل تعريف عدة واجهات صغيرة ومتخصصة بدل واجهة واحدة عملاقة، بحيث محدش يضطر يعتمد على Methods مش محتاجها أصلًا.

١. ليه مبدأ ISP مهم؟

لما واجهة (Interface) بتحتوي على عدد كبير من الـ Methods غير المترابطة، أي كلاس بيطبق الواجهة دي بيتجبر ينفذ Methods مش محتاجها أصلًا، وده بيزود الاعتماد غير الضروري (Unnecessary Coupling) ويصعب فهم الغرض الحقيقي من الكلاس.

في مشاريع ML، ده بيظهر لما تحاول تعرف واجهة واحدة شاملة لكل أنواع النماذج، لكن مش كل نموذج محتاج نفس المجموعة الكاملة من الوظائف (زي بعض النماذج مش محتاجة save_checkpoint أثناء التدريب).

٢. المشكلة اللي بيحلها ISP

المشكلة هي الواجهات الضخمة (Fat Interfaces) اللي بتجبر أي تطبيق ليها ينفذ Methods كتير مش كلها مهمة له، وأحيانًا بيضطر المهندس يكتب تطبيق فارغ أو يرمي استثناء NotImplementedError لكل Method مش محتاجها، وده بيخالف مبدأ LSP كمان لأن السلوك بقى غير متوقع.

٣. إزاي كانت الشركات بتتعامل قبل تبني ISP

كان شائع تعريف واجهة أو كلاس أساسي واحد شامل يحتوي على كل الوظائف الممكنة لمفهوم معين (زي واجهة Model واحدة فيها كل Method ممكن يحتاجه أي نوع نموذج)، بحجة توحيد التعامل مع كل الأنواع من مكان واحد.

٤. ليه الطريقة القديمة مش مناسبة

الواجهة الشاملة بتخلق اعتماد وهمي بين كلاسات ملهاش علاقة ببعض فعليًا، وأي تعديل في الواجهة الكبيرة دي (زي إضافة Method جديد) بيجبر كل التطبيقات الموجودة تتعدل، حتى لو الـ Method الجديد ده مش له علاقة بمعظمها.

٥. الفكرة الأساسية وراء ISP

بدل واجهة واحدة كبيرة، بتقسمها لعدة واجهات صغيرة متخصصة، كل واحدة بتمثل مجموعة وظائف مترابطة منطقيًا. أي كلاس بيطبق بس الواجهات اللي فعلًا محتاجها، من غير التزام بوظائف مش له علاقة بيها.

٦. المفاهيم الأساسية

- Fat Interface: واجهة تحتوي على عدد كبير من الوظائف غير المترابطة، وهي المشكلة اللي ISP بيحاول يتجنبها.
- Role Interfaces: تصميم واجهات صغيرة حسب الدور الوظيفي (زي Trainable، Predictable، Persistable) بدل واجهة واحدة شاملة.
- Client-Specific Interfaces: تصميم الواجهة حسب احتياج كل مستخدم (Client) لها، مش حسب كل الوظائف الممكنة نظريًا.

٧. المكونات

- تحليل الاحتياجات الفعلية لكل نوع من مستخدمي الواجهة.
- تقسيم الواجهة الكبيرة لعدة واجهات صغيرة حسب الأدوار المنطقية.
- كل كلاس بيطبق مجموعة الواجهات الصغيرة اللي فعلًا محتاجها.

٨. إزاي المبدأ ده بيشتغل داخليًا

لما تقسم الواجهة لأجزاء صغيرة، أي تعديل في جزء معين بيأثر بس على الكلاسات اللي فعلًا بتستخدم الجزء ده، وباقي الكلاسات مش بتتأثر خالص. ده بيقلل نطاق التأثير (Blast Radius) لأي تغيير مستقبلي في الواجهات.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في تصميم أنظمة الـ ML Platforms الكبيرة، بيتم تعريف واجهات صغيرة منفصلة زي Trainable وServable وCheckpointable، وكل نموذج بيطبق بس الواجهات اللي فعلًا يحتاجها. النموذج اللي بيتقدم للمستخدمين بس (Serving Only) مش محتاج يطبق واجهة Trainable مثلًا.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في تصميم الواجهات اللي بتحدد سلوك النماذج والمعالجات المختلفة، خصوصًا لما يكون فيه تنوع كبير في احتياجات كل نوع (بعض النماذج تحتاج checkpointing، بعضها لأ، بعضها يحتاج batch prediction، بعضها بس single prediction).

١١. امتى تطبق ISP

- لما تلاحظ إن واجهة معينة بتحتوي على وظائف مش كل مستخدميها محتاجينها.
- لما تلاحظ تطبيقات فارغة أو استثناءات NotImplementedError متكررة في تطبيقات الواجهة.

١٢. امتى ممكن تسيب واجهة واحدة موحدة

- لما كل الحالات فعلًا بتحتاج نفس مجموعة الوظائف بالكامل من غير استثناء.
- لما التقسيم المفرط للواجهات هيزود التعقيد من غير فايدة حقيقية.

١٣. أفضل الممارسات

Best Practices

- قسم الواجهات الكبيرة حسب الأدوار الوظيفية المنطقية بدل الاحتفاظ بواجهة شاملة واحدة.
- لاحظ التطبيقات الفارغة أو الاستثناءات NotImplementedError كعلامة على واجهة تحتاج تقسيم.
- صمم الواجهات بناءً على احتياجات المستخدمين الفعليين لها، مش كل الوظائف الممكنة نظريًا.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- تعريف واجهة عملاقة واحدة تحاول تغطي كل الحالات الممكنة.
- ترك Methods فاضية أو بترمي NotImplementedError في التطبيقات اللي مش محتاجاها.
- إجبار كل الكلاسات الجديدة تطبق نفس الواجهة الكبيرة الموجودة، حتى لو معظمها مش مناسب لها.

١٥. اعتبارات الأداء

تقسيم الواجهات لأجزاء صغيرة مالوش أي تأثير سلبي على الأداء وقت التشغيل، لأنه مجرد إعادة تنظيم على مستوى التصميم، مش تغيير في المنطق الفعلي أو طريقة التنفيذ.

١٦. الاعتبارات الأمنية

الواجهات الصغيرة والمحددة بتسهل تحديد نطاق الصلاحيات (Permissions) المطلوبة لكل جزء من النظام، بدل منح صلاحيات واسعة غير ضرورية لكلاس بس بيستخدم جزء صغير من واجهة كبيرة.

١٧. اعتبارات قابلية التوسع

الواجهات الصغيرة والمتخصصة بتسمح بإضافة أنواع جديدة تطبق بس الأجزاء اللي محتاجينها، من غير الحاجة لتطبيق واجهة ضخمة بالكامل، وده بيسهل ويسرع عملية إضافة أنواع جديدة للنظام.

١٨. مقارنة مع نهج بديل

البديل هو واجهة شاملة واحدة توفر تناسق ظاهري بين كل الأنواع. الأسلوب ده بيبدو أبسط في البداية، لكنه بيخلق اعتماد وهمي وتعقيد غير ضروري مع تنوع الحالات. الشركات الكبرى بتفضل الواجهات الصغيرة المتخصصة رغم الحاجة لتنظيم أكتر في البداية.

١٩. مثال عملي صغير

```
# Before - one fat interface forcing unrelated methods
class Model(ABC):
    @abstractmethod
    def train(self, data): ...
    @abstractmethod
    def predict(self, data): ...
    @abstractmethod
    def save_checkpoint(self, path): ...

# After - small, role-specific interfaces
class Trainable(ABC):
    @abstractmethod
    def train(self, data): ...

class Predictable(ABC):
    @abstractmethod
    def predict(self, data): ...

class Checkpointable(ABC):
    @abstractmethod
    def save_checkpoint(self, path): ...

# A serving-only model implements only what it needs
class ServingOnlyModel(Predictable):
    def predict(self, data):
        ...
```

٢٠. مثال من مشروع Production حقيقي

في منصة ML كبيرة، النماذج اللي بتتقدم للمستخدمين النهائيين بس (Inference Servers) بتطبق واجهة Predictable الصغيرة، بينما النماذج اللي لسه في مرحلة التطوير بتطبق كمان Trainable وCheckpointable. الفصل ده بيمنع كود الـ Serving من الاعتماد على وظائف تدريب مش موجودة أو غير ضرورية في بيئة الإنتاج.

٢١. ملخص الموضوع

خلاصة ISP

- فضل واجهات صغيرة متخصصة على واجهة واحدة شاملة.
- محدش المفروض يعتمد على Methods مش محتاجها فعلًا.
- التطبيقات الفارغة أو استثناءات NotImplementedError علامة واضحة على واجهة محتاجة تقسيم.

Dependency Inversion Principle (DIP) ٢,٥

المبدأ الأخير في SOLID، وواحد من أقوى المبادئ تأثيرًا على بنية النظام كله: الوحدات عالية المستوى (High-Level Modules) ميفروضش تعتمد على الوحدات منخفضة المستوى (Low-Level Modules) مباشرة، الاتنين لازم يعتمدوا على تجريد (Abstraction) مشترك.

١. ليه مبدأ DIP مهم؟

لو منطق الأعمال الرئيسي (زي Pipeline التدريب) بيعتمد مباشرة على تفاصيل تقنية محددة (زي نوع قاعدة بيانات معينة أو مكتبة تخزين معينة)، أي تغيير في التفاصيل التقنية دي بيجبرك تعدل في منطق الأعمال الأساسي، رغم إن منطق الأعمال نفسه ميتغيرش. DIP بيحل المشكلة دي عن طريق عكس اتجاه الاعتماد.

في مشاريع ML، ده بيسمح لمنطق التدريب الأساسي إنه يفضل ثابت حتى لو غيرت مصدر البيانات من ملفات CSV لقاعدة بيانات أو لـ Data Warehouse.

٢. المشكلة اللي بيحلها DIP

المشكلة هي الاعتماد المباشر للمنطق عالي المستوى (زي منطق التدريب) على تفاصيل تقنية منخفضة المستوى (زي مكتبة معينة لقراءة الملفات). لو غيرت المكتبة أو طريقة التخزين، لازم تعدل في كل مكان بيستخدم المنطق عالي المستوى ده، حتى لو المنطق نفسه ميتغيرش.

٣. إزاي كانت الشركات بتتعامل قبل تبني DIP

كان شائع كتابة كود بيستدعي مكتبات وتفاصيل تقنية محددة مباشرة جوه منطق الأعمال (زي استدعاء pandas.read_csv مباشرة جوه دالة التدريب)، من غير طبقة فاصلة بين منطق الأعمال والتفاصيل التقنية.

٤. ليه الطريقة القديمة مش مناسبة

الاعتماد المباشر على التفاصيل التقنية بيخلي منطق الأعمال هش وصعب الاختبار، لأن اختبار منطق التدريب مثلًا بيحتاج ملف CSV حقيقي بدل بيانات وهمية بسيطة (Mock Data)، وأي تغيير في مصدر البيانات بيجبرك تعدل منطق الأعمال نفسه.

٥. الفكرة الأساسية وراء DIP

بدل ما منطق الأعمال يعتمد مباشرة على تفاصيل تقنية معينة، بتعرف واجهة مجردة (Abstraction) زي DataSource، ومنطق الأعمال بيعتمد على الواجهة دي بس. التفاصيل التقنية (زي القراءة من CSV أو قاعدة بيانات) بتتنفذ كتطبيق منفصل للواجهة دي، وده بيعكس اتجاه الاعتماد: التفاصيل بقت هي اللي بتعتمد على التجريد، مش العكس.

٦. المفاهيم الأساسية

- High-Level vs Low-Level Modules: المنطق الأساسي للأعمال (High-Level) مقابل التفاصيل التقنية (Low-Level) زي قواعد البيانات والمكتبات الخارجية.
- Dependency Injection: تقنية عملية لتطبيق DIP، حيث بتمرر التطبيق الفعلي للواجهة (Concrete Implementation) للكلاس من برا بدل ما الكلاس ينشئه بنفسه.
- Inversion of Control: المبدأ الأوسع اللي DIP جزء منه، حيث التحكم في اختيار التفاصيل التقنية بينتقل من الكود نفسه لمصدر خارجي (زي Configuration أو Framework).

٧. المكونات

- واجهة مجردة (Abstraction) بتمثل الخدمة المطلوبة (زي DataSource أو NotificationService).
  - تطبيقات فعلية (Concrete Implementations) للواجهة دي، كل واحدة بتفاصيل تقنية مختلفة.
- آلية حقن الاعتماد (Dependency Injection) بتمرر التطبيق المطلوب للكلاس اللي محتاجه، عادة عن طريق الـ Constructor.

٨. إزاي المبدأ ده بيشتغل داخليًا

لما الكلاس بيستقبل الاعتماد بتاعه (Dependency) عن طريق الـ Constructor بدل ما ينشئه بنفسه جوه، فهو بيصبح غير مرتبط بتفصيل تقني محدد. وقت الاختبار، تقدر تمرر تطبيق وهمي (Mock) للواجهة بدل التطبيق الحقيقي، وده بيخلي الاختبار أسرع وأبسط بكتير من غير الاعتماد على موارد خارجية حقيقية.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في أنظمة الـ ML الكبيرة، منطق التدريب الأساسي غالبًا بيعتمد على واجهة مجردة لمصدر البيانات (DataSource Interface)، والتطبيق الفعلي (قراءة من S3، أو من Data Warehouse، أو من قاعدة بيانات) بيتحدد وقت التشغيل عن طريق ملف Configuration، من غير أي تعديل في منطق التدريب نفسه.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر بوضوح في الفصل بين منطق التدريب ومصدر البيانات، وبين منطق التنبؤ وطريقة تخزين النموذج، وبين منطق الأعمال وأي خدمة خارجية زي إرسال إشعارات أو تسجيل الأحداث (Logging).

١١. امتى تطبق DIP

- لما منطق الأعمال الأساسي محتاج يفضل مستقر حتى لو التفاصيل التقنية (قواعد بيانات، مكتبات) اتغيرت.
- لما محتاج تختبر منطق الأعمال بمعزل عن الموارد الخارجية الحقيقية (قواعد بيانات، شبكة، ملفات).

١٢. امتى DIP ممكن يكون زيادة عن اللزوم

- في سكربتات صغيرة جدًا ومؤقتة مش متوقع ليها أي تغيير في التفاصيل التقنية.
- لما إضافة طبقة تجريد كاملة هتزود التعقيد من غير فايدة حقيقية للمشروع الصغير.

١٣. أفضل الممارسات

Best Practices

- خلي منطق الأعمال الأساسي يعتمد على واجهات مجردة، مش على تفاصيل تقنية محددة مباشرة.
- استخدم Dependency Injection لتمرير التطبيقات الفعلية للاعتماديات عن طريق الـ Constructor.
- اكتب اختبارات باستخدام تطبيقات وهمية (Mocks) للواجهات المجردة بدل الموارد الحقيقية.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- استدعاء مكتبات أو خدمات خارجية مباشرة جوه منطق الأعمال الأساسي من غير طبقة تجريد.
- إنشاء الاعتماديات (Dependencies) جوه الكلاس نفسه بدل حقنها من برا.
- المبالغة في استخدام DIP لحالات بسيطة جدًا مش متوقع ليها أي تغيير مستقبلي (Over-engineering).

١٥. اعتبارات الأداء

طبقة التجريد الإضافية بتضيف تكلفة بسيطة جدًا في استدعاء الدوال، لكنها مهملة عمليًا. الفايدة في المرونة وسهولة الاختبار أكبر بكتير من أي تكلفة أداء طفيفة.

١٦. الاعتبارات الأمنية

فصل منطق الأعمال عن التفاصيل التقنية بيسمح بتطبيق سياسات أمان مختلفة على التطبيقات الفعلية (زي التحقق من الصلاحيات في تطبيق معين لقراءة البيانات) من غير التأثير على منطق الأعمال الأساسي أو الحاجة لتكراره في كل تطبيق.

١٧. اعتبارات قابلية التوسع

DIP بيسمح بإضافة تطبيقات جديدة لمصادر بيانات أو خدمات خارجية جديدة من غير التأثير على منطق الأعمال الأساسي، وده بيسهل التوسع لدعم بيئات أو تقنيات جديدة مع نمو المشروع.

١٨. مقارنة مع نهج بديل

البديل هو الاعتماد المباشر على التفاصيل التقنية من غير طبقة تجريد، وهو أسرع في الكتابة الأولى لمشروع صغير جدًا. لكن الشركات الكبرى بتفضل الاستثمار في DIP من البداية لأي نظام متوقع له عمر طويل أو تغييرات تقنية مستقبلية، لأن تكلفة إعادة الهيكلة لاحقًا أكبر بكتير من تكلفة التصميم الصح من الأول.

١٩. مثال عملي صغير

```
# Before - training logic directly depends on a specific library
class ModelTrainer:
    def load_data(self, path):
        return pandas.read_csv(path)

    def train(self, path):
        data = self.load_data(path)
        # training logic here

# After - training logic depends on an abstraction, injected from outside
from abc import ABC, abstractmethod

class DataSource(ABC):
    @abstractmethod
    def load(self):
        ...

class CsvDataSource(DataSource):
    def __init__(self, path):
        self.path = path
    def load(self):
        return pandas.read_csv(self.path)

class ModelTrainer:
    def __init__(self, data_source: DataSource):
        self.data_source = data_source

    def train(self):
        data = self.data_source.load()
        # training logic here, unaware of the data's real source
```

٢٠. مثال من مشروع Production حقيقي

في نظام تدريب حقيقي بيشتغل على منصة سحابية، منطق التدريب بيعتمد على واجهة مجردة FeatureStore، والتطبيق الفعلي بيتغير حسب البيئة: بيئة التطوير بتستخدم تطبيق يقرأ من ملفات محلية، وبيئة الإنتاج بتستخدم تطبيق يقرأ من نظام Feature Store سحابي حقيقي، من غير أي تغيير في منطق التدريب نفسه.

٢١. ملخص الموضوع

خلاصة DIP

- منطق الأعمال عالي المستوى لازم يعتمد على تجريد، مش على تفاصيل تقنية محددة.
- استخدم Dependency Injection لتمرير التطبيقات الفعلية من برا.
- الفايدة الأكبر: منطق أعمال مستقر وسهل الاختبار، حتى لو التفاصيل التقنية اتغيرت بالكامل.

مشروع عملي على الفصل: إعادة هيكلة نظام تدريب نماذج بمبادئ SOLID

المشروع ده هدفه إنك تطبق الخمس مبادئ (SRP، OCP، LSP، ISP، DIP) على نظام تدريب نماذج بسيط، بحيث تشوف إزاي المبادئ الخمسة بتتكامل مع بعض في تصميم واحد متماسك.

السيناريو: عندك كلاس واحد اسمه MLSystem بيعمل كل حاجة: بيقرأ بيانات من ملف CSV مباشرة، بيدرب نموذج Linear Regression بس (من غير إمكانية إضافة نوع تاني)، بيحفظ النتيجة في ملف، وبيبعت إيميل تقرير. الكلاس ده بيخالف كل مبادئ SOLID في نفس الوقت.

خطوات التنفيذ

- طبق SRP: افصل الكلاس لأربع كلاسات منفصلة: ModelRepository، ModelTrainer، DataLoader، TrainingReporter.
- طبق OCP: عرف واجهة ModelStrategy مجردة، واخلي إضافة أنواع نماذج جديدة تتم بإضافة كلاس جديد بس، من غير تعديل منطق التدريب.
- طبق LSP: تأكد إن كل تطبيق لـ ModelStrategy بيلتزم بنفس شكل المدخلات والمخرجات المتوقع، من غير استثناءات أو سلوك غير متوقع.
- طبق ISP: لو فيه نماذج مش محتاجة كل الوظائف (زي checkpointing)، قسم الواجهة لواجهات أصغر حسب الحاجة الفعلية.
- طبق DIP: خلي كلاس ModelTrainer يعتمد على واجهة DataSource مجردة بدل الاعتماد المباشر على pandas.read_csv، وحقن التطبيق الفعلي من برا.

كل المفاهيم الخمسة مرتبطة علي بعض solid خدبالك ان

o: اول حاجه انت هتفصل الكلاسات عن بعض وانت لما تفصل الكلاسات محتاج تعرف تتعامل معاهم يجي هنا دور s: لو عدلت في واحد اللي بيعتمد عليه ميتأثرش وهنا فيه حاجه برضو هما لما قسمنهاهم فيه كلاسات منفصلة وفي كلاسات مينفش تورث من كلاس انت مختلف عنه في التصرفات وفي الحالة دي l: زي بعض بتتعمل وراثة بس في الحالة دي مينفعش تورث حاجات انت مش محتاجه فانت محتاج تقسم i: بقي لو انت نفس التصرفات كلها وبتورث كل التصرفات الكلاس وتورث فقط اللي انت هتستخدمه ويجي هنا مشكلة كمان انت لما قسمت الكلاسات منفصلة عندك في الكلاس انت هنا محتاج تقسم d: الواحد حاجات متغيرة انت مثبت الكود بتاعها زي التعامل مع البيانات انت محدد نوع واحد بس الكلاس لجزئين جزء الحاجة المتغيرة دي والجزء بتاع الكلاس نفسه والكلاس الحاجة المتغيرة ده محتاج تتعامل معاه اللي هو أي تعديل ميأثرش علي الكلاس اللي معتمد عليه o بنفس منطق

تحديات شائعة عند التطبيق

- معرفة متى تتوقف: تطبيق SOLID بشكل مطلق على كل جزء من الكود، حتى لو بسيط جدًا، بيؤدي لتعقيد غير ضروري (Over-engineering). القاعدة العملية: طبق المبدأ لما فيه توقع حقيقي للتغيير أو التوسع المستقبلي.
- الخلط بين OCP وISP: المبدأين بيستخدموا التجريد (Abstraction) بشكل مشابه، لكن OCP بيركز على إضافة سلوك جديد من غير تعديل، بينما ISP بيركز على تقسيم الواجهات حسب الاحتياج الفعلي. من المفيد تسأل نفسك: هل المشكلة إني محتاج أضيف نوع جديد (OCP)، ولا إني مجبر أنفذ وظائف مش محتاجها (ISP)؟
- تطبيق DIP من غير فهم حقيقي للفايدة: بعض المهندسين بيضيفوا طبقة تجريد (Abstraction Layer) لكل حاجة بشكل آلي من غير التفكير في السبب، وده بيزود تعقيد الكود من غير فايدة حقيقية. الهدف من DIP هو المرونة وسهولة الاختبار، مش التجريد لمجرد التجريد.

إزاي الفصل ده بيتستخدم داخل مشروع Machine Learning حقيقي

مبادئ SOLID بتظهر في كل قرار تصميمي كبير في مشروع ML حقيقي: لما تفصل مسؤوليات الـ Pipeline (SRP)، لما تصمم نظام يدعم أنواع نماذج متعددة من غير تعديل الكود الأساسي (OCP)، لما تتأكد إن كل نموذج جديد بيتوافق مع الواجهة المتوقعة (LSP)، لما تصمم واجهات صغيرة متخصصة للنماذج المختلفة (ISP)، ولما تخلي منطق التدريب مستقل عن مصدر البيانات الفعلي (DIP).

في الشركات الكبرى، فرق الـ ML Platform بتصمم بنية تحتية كاملة مبنية على مبادئ SOLID، عشان تسمح لفرق النمذجة المختلفة يضيفوا نماذج وخوارزميات جديدة بسرعة وأمان، من غير الحاجة لفهم أو تعديل الكود الأساسي للمنصة في كل مرة.

ملخص الفصل الثاني وربطه بالفصل القادم

في الفصل ده اتعلمت خمس مبادئ بتحكم إزاي تصمم كلاسات وعلاقاتها ببعض بشكل مرن وقابل للتوسع: SRP (مسؤولية واحدة لكل كلاس)، OCP (مفتوح للتوسع مغلق للتعديل)، LSP (الكلاس الفرعي لازم يحل محل الأساسي بأمان)، ISP (واجهات صغيرة متخصصة بدل واجهة شاملة)، وDIP (الاعتماد على التجريد مش على التفاصيل التقنية).

الفصل الجاي (Design Patterns) هيبني مباشرة على المبادئ دي. كل نمط تصميم هنشرحه - Singleton، Factory، Dependency Injection، Observer، Strategy - هو في الحقيقة تطبيق عملي جاهز لواحد أو أكتر من مبادئ SOLID على مشكلة تصميم متكررة وشائعة. مثلًا، نمط Strategy هو تطبيق مباشر لمبدأ OCP، ونمط Dependency Injection هو الأسلوب العملي لتطبيق DIP. لما توصل للفصل الجاي، هتلاحظ إنك بتتعرف على أدوات جاهزة لمشاكل فهمت جذورها بالفعل هنا.